

Data Collection & Cleaning

STEP 1 : Import & Load Data

```
In [1]: import pandas as pd

df = pd.read_csv("synthetic_retail_dataset.csv")
```

STEP 2 : Checking Data Shape and Columns and data types

```
In [2]: print(df.shape)      # rows and columns
print(df.columns)          # column names
print(df.info())           # data types + missing values

(1050, 8)
Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'UnitPrice',
      'CustomerID', 'Country', 'Date'],
      dtype='object')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1050 entries, 0 to 1049
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   InvoiceNo        1050 non-null  int64   
1   StockCode       832 non-null   object  
2   Description      875 non-null   object  
3   Quantity        1016 non-null  float64  
4   UnitPrice       1000 non-null  float64  
5   CustomerID      838 non-null   float64  
6   Country         879 non-null   object  
7   Date            1050 non-null  object  
dtypes: float64(3), int64(1), object(4)
memory usage: 65.8+ KB
None
```

```
In [3]: df.head()
```

Out[3]:

	InvoiceNo	StockCode	Description	Quantity	UnitPrice	CustomerID	Country	Date
0	1046	NaN	Monitor	NaN	475.08	104.0	India	2010-01-02 02:00
1	1868	A101	Mouse	0.0	905.72	104.0	USA	2010-02-08 08:00
2	1452	C303	Mouse	-2.0	115.19	104.0	usa	2010-01-23 23:00
3	1098	A101	Monitor	19.0	850.17	101.0	USA	2010-02-12 12:00
4	1619	C303	Mouse	7.0	3.01	NaN	UK	2010-01-01 01:00

STEP 3 : Handling Missing Values

In []:

In [4]: `num_cols = df.select_dtypes(include=['int64', 'float64']).columns`

In [5]: `print("Missing Values BEFORE cleaning:\n")`
`print(df.isnull().sum())`

Missing Values BEFORE cleaning:

```
InvoiceNo      0
StockCode     218
Description    175
Quantity       34
UnitPrice      50
CustomerID    212
Country       171
Date           0
dtype: int64
```

In [6]: `df = df.dropna(subset=['CustomerID'])`

In [7]: `df.shape`

Out[7]: (838, 8)

In [8]: `# Numerical → Mean`
`#num_cols = df.select_dtypes(include=['int64', 'float64']).columns`
`#df[num_cols] = df[num_cols].fillna(df[num_cols].mean())`

`#Numerical → Mean`
`num_cols = df.select_dtypes(include=['int64', 'float64']).columns`

```
# Remove Customer ID Column
num_cols = num_cols.drop('CustomerID')

for col in num_cols:
    df[col] = df[col].fillna(df[col].mean())

# Categorical → Mode
cat_cols = df.select_dtypes(include=['object']).columns
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```
In [9]: print("Missing Values AFTER cleaning:\n")
        print(df.isnull().sum())
```

Missing Values AFTER cleaning:

```
InvoiceNo      0
StockCode      0
Description    0
Quantity       0
UnitPrice      0
CustomerID     0
Country        0
Date           0
dtype: int64
```

Step 5 : Check Duplicates Rows

```
In [10]: print("Duplicate Rows BEFORE:", df.duplicated().sum())
```

Duplicate Rows BEFORE: 40

```
In [11]: df = df.drop_duplicates()
```

```
In [12]: print("Duplicate Rows AFTER:", df.duplicated().sum())
```

Duplicate Rows AFTER: 0

```
In [ ]:
```

Step 6 : Check Inconsistent Text

```
In [13]: print("Unique Country Values BEFORE:\n")
        print(df['Country'].unique())
```

Unique Country Values BEFORE:

```
['India' 'USA' 'usa' ' India' 'UK']
```

```
In [14]: # Remove spaces
        df['Country'] = df['Country'].str.strip()

        # Convert to uppercase
        df['Country'] = df['Country'].str.upper()

        # Clean Description
        df['Description'] = df['Description'].str.strip()
```

```
In [15]: print("Unique Country Values AFTER:\n")
print(df['Country'].unique())
```

Unique Country Values AFTER:

['INDIA' 'USA' 'UK']

```
In [16]: df.head(5)
```

Out[16]:

	InvoiceNo	StockCode	Description	Quantity	UnitPrice	CustomerID	Country	
0	1046	A101	Monitor	20.378713	475.08	104.0	INDIA	
1	1868	A101	Mouse	0.000000	905.72	104.0	USA	
2	1452	C303	Mouse	-2.000000	115.19	104.0	USA	
3	1098	A101	Monitor	19.000000	850.17	101.0	USA	
5	1518	D404	Laptop	48.000000	579.76	101.0	USA	

```
In [17]: df[df['Quantity']<0]
```

Out[17]:

	InvoiceNo	StockCode	Description	Quantity	UnitPrice	CustomerID	Country		
	2	1452	C303	Mouse	-2.0	115.19	104.0	USA	2
	8	1980	A101	Monitor	-7.0	661.01	102.0	USA	0
	11	1769	A101	Monitor	-1.0	994.38	104.0	USA	2
	17	1386	C303	Monitor	-7.0	-44.00	103.0	USA	1
	18	1612	D404	Keyboard	-1.0	203.98	101.0	UK	0
	...	...	...	...	...	...	...	...	
	985	1797	A101	Mouse	-3.0	633.51	102.0	USA	1
	1010	1004	B202	Mouse	-4.0	-17.01	102.0	USA	1
	1028	1841	A101	Mouse	-8.0	778.90	103.0	UK	1
	1032	1098	A101	Mouse	-5.0	873.47	101.0	USA	1
	1038	1421	B202	Monitor	-5.0	509.91	104.0	USA	1

117 rows × 8 columns



Step 7 : Check Invalid Values

```
In [18]: print("Negative Quantity Count:", (df['Quantity'] < 0).sum())
print("Negative Price Count:", (df['UnitPrice'] < 0).sum())
```

Negative Quantity Count: 117
Negative Price Count: 44

```
In [19]: df = df[df['Quantity'] >= 0]
df = df[df['UnitPrice'] >= 0]
```

```
In [20]: print("Negative Quantity Count:", (df['Quantity'] < 0).sum())
```

```
print("Negative Price Count:", (df['UnitPrice'] < 0).sum())
```

Negative Quantity Count: 0

Negative Price Count: 0

Step 8 : Convert Date format

```
In [21]: df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
```

```
In [22]: print(df.columns)
```

```
Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'UnitPrice',  
      'CustomerID', 'Country', 'Date'],  
      dtype='object')
```

Step 9: Standardize Column Names

```
In [23]: df.columns = df.columns.str.lower().str.replace(" ", "_")  
print(df.columns)
```

```
Index(['invoiceno', 'stockcode', 'description', 'quantity', 'unitprice',  
      'customerid', 'country', 'date'],  
      dtype='object')
```

Step 10 : Final Check

```
In [24]: print("Final Shape:", df.shape)  
  
print("\nFinal Info:\n")  
print(df.info())  
  
print("\nPreview:\n")  
print(df.head())
```

Final Shape: (646, 8)

Final Info:

```
<class 'pandas.core.frame.DataFrame'>
Index: 646 entries, 0 to 1049
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   invoiceno        646 non-null    int64
1   stockcode        646 non-null    object
2   description       646 non-null    object
3   quantity         646 non-null    float64
4   unitprice        646 non-null    float64
5   customerid       646 non-null    float64
6   country          646 non-null    object
7   date             646 non-null    datetime64[ns]
dtypes: datetime64[ns](1), float64(3), int64(1), object(3)
memory usage: 45.4+ KB
None
```

Preview:

	invoiceno	stockcode	description	quantity	unitprice	customerid	country	\
0	1046	A101	Monitor	20.378713	475.08	104.0	INDIA	
1	1868	A101	Mouse	0.000000	905.72	104.0	USA	
3	1098	A101	Monitor	19.000000	850.17	101.0	USA	
5	1518	D404	Laptop	48.000000	579.76	101.0	USA	
6	1053	C303	Mouse	12.000000	320.23	101.0	USA	

	date
0	2023-01-28 02:00:00
1	2023-02-10 08:00:00
3	2023-02-01 12:00:00
5	2023-02-05 01:00:00
6	2023-01-17 00:00:00

Step 11 : Save Cleaned Dataset

```
In [25]: df.to_csv("cleaned_retail_dataset.csv", index=False)
```

```
print("✅ Cleaned dataset saved")
```

✅ Cleaned dataset saved

```
In [ ]:
```